

Web Technologies



WET2VO

Teil 1: Grundlagen der Kommunikation im Web

Medientechnik und -design | SS 2026

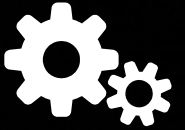
Organisatorisches

Hypermedia 2 Ablauf



12

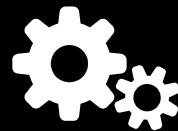
**Vorlesungen á 90 Minuten
Abschließende e-Klausur**



12

Übungen

6 Abgaben, 6 code-along



RDB2VO

Grundlagen der Datenbanken
werden in der LVA verwendet



Skriptum

Im eLearning-Kurs vor der Vorlesung
Slides auf Wunsch auch dabei

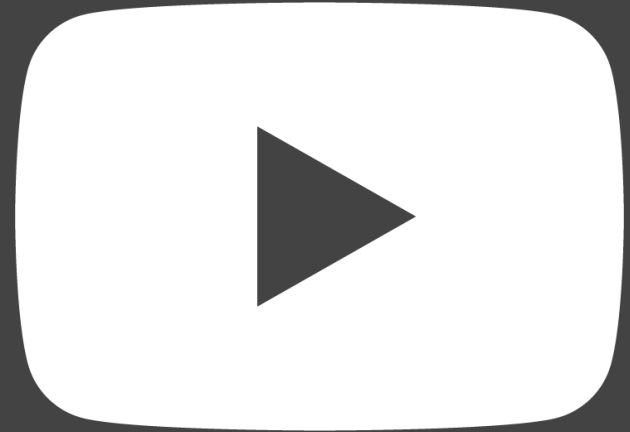




Livestream auf **YouTube** aus dem Hörsaal

Fragen übers Fragetool

Chat im Hörsaal nicht verwendbar



Themenfelder

Web Technologies Inhalte



Komponenten

Templates

Serverseitige Entwicklung

Regular Expressions

Strings

Medienverarbeitung

Microframeworks

Routing

REST APIs

OOP

Sessions

AJAX

Testing

PHP

Formularverarbeitung

Internationalisierung

Datum & Zeit

Netzwerkgrundlagen

Webserver

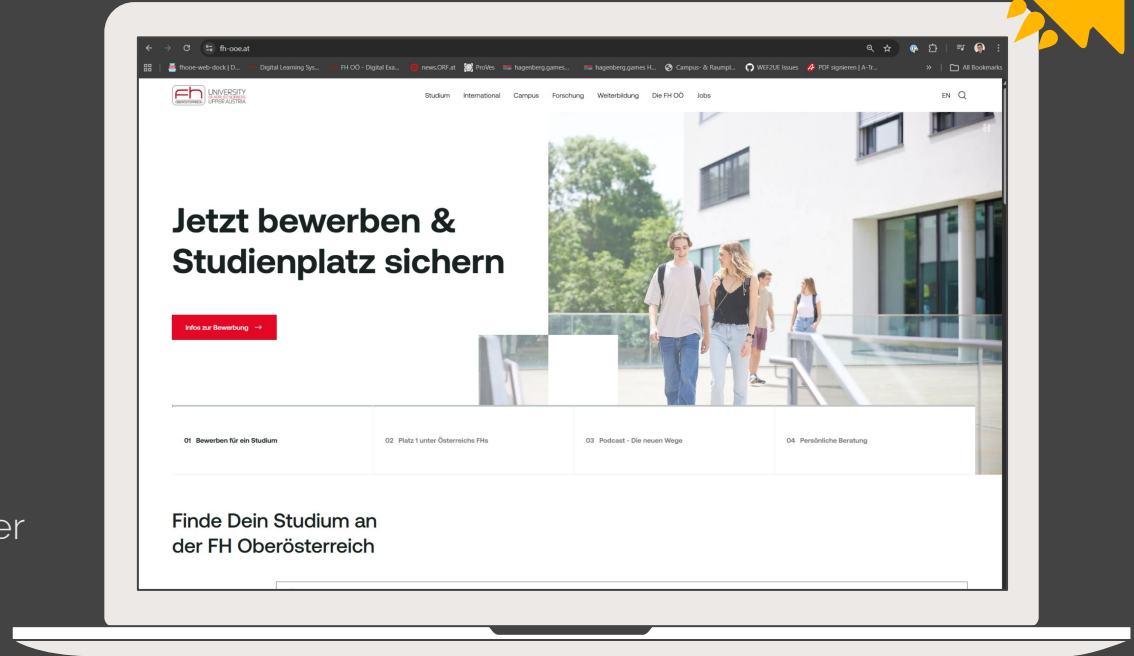
Der Webrequest

Unser Leitmotiv für die nötigen Netzwerkgrundlagen

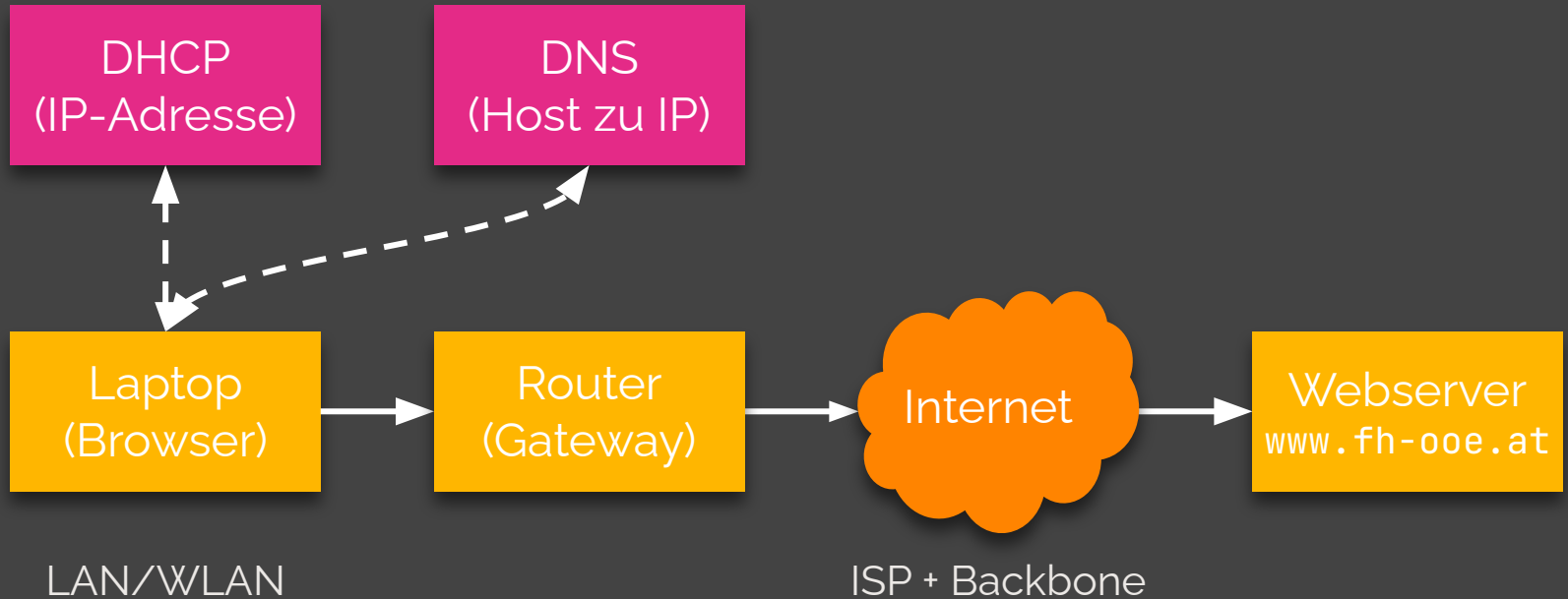
Webseite im Browser öffnen

Was passiert, wenn wir
<https://www.fh-ooe.at/> im
Browser eintippen und Enter
drücken?

Ein Kommunikationsablauf
über das Netzwerk!



Ablauf vereinfacht



Schritte im Detail



#	Aktion	Initiator	Protokoll/Dienst
1.	URL www.fh-ooe.at zerlegen	Browser	-
2.	Laptop benötigt eine IP-Adresse im Netz	Laptop	DHCP
3.	Hostname www.fh-ooe.at muss in eine IP-Adresse übersetzt werden	Browser	DNS
4.	Hardware-Adresse (MAC) des Routers/Gateways erfragen	OS	ARP
5.	TCP-Verbindung zum Webserver auf Port 443 (HTTPS) aufbauen*	Browser	TCP
6.	Verschlüsselten Kanal per TLS verhandeln*	Browser	TLS
7.	HTTP-Request verschicken*	Browser	HTTP
8.	Request verarbeiten (z.B. mit PHP) und HTTP-Response schicken*	Webserver	HTTP
9.	HTML parsen und Anzeigen	Browser	-

* Daten werden vom sendenden Rechner in TCP-, IP- und Ethernet-Header verpackt (Encapsulation) und vom empfangenden Rechner wieder ausgepackt

Der URL

Der Uniform Resource Locator als Adressierungsschema



URL-Bestandteil

e <schema>: <schemaspezifischer Teil>

Schemata: http, https, file, mailto, ftp, ws, wss, ...

`https://user:pass@www.example.org:8080/shop/item.php?id=42&lang=de#top`

- `https://` Schema (HTTP über TLS verschlüsselt)
- `user:pass@` Optionale Zugangsdaten für HTTP-Basic-Auth (veraltet)
- `www.example.org` Host (hier als Fully Qualified Domain Name, FQDN)
- `:8080` Port auf dem der Webserver hört (80: HTTP, 443: HTTPS)
- `/shop/item.php` Pfad der Ressource
- `?id=42&lang=de` Query-String (Daten, die beim Request mitgegeben werden)
- `#top` Fragment (Abschnitt innerhalb der Ressource)



FH OÖ-URL

`https://www.fh-ooe.at/`

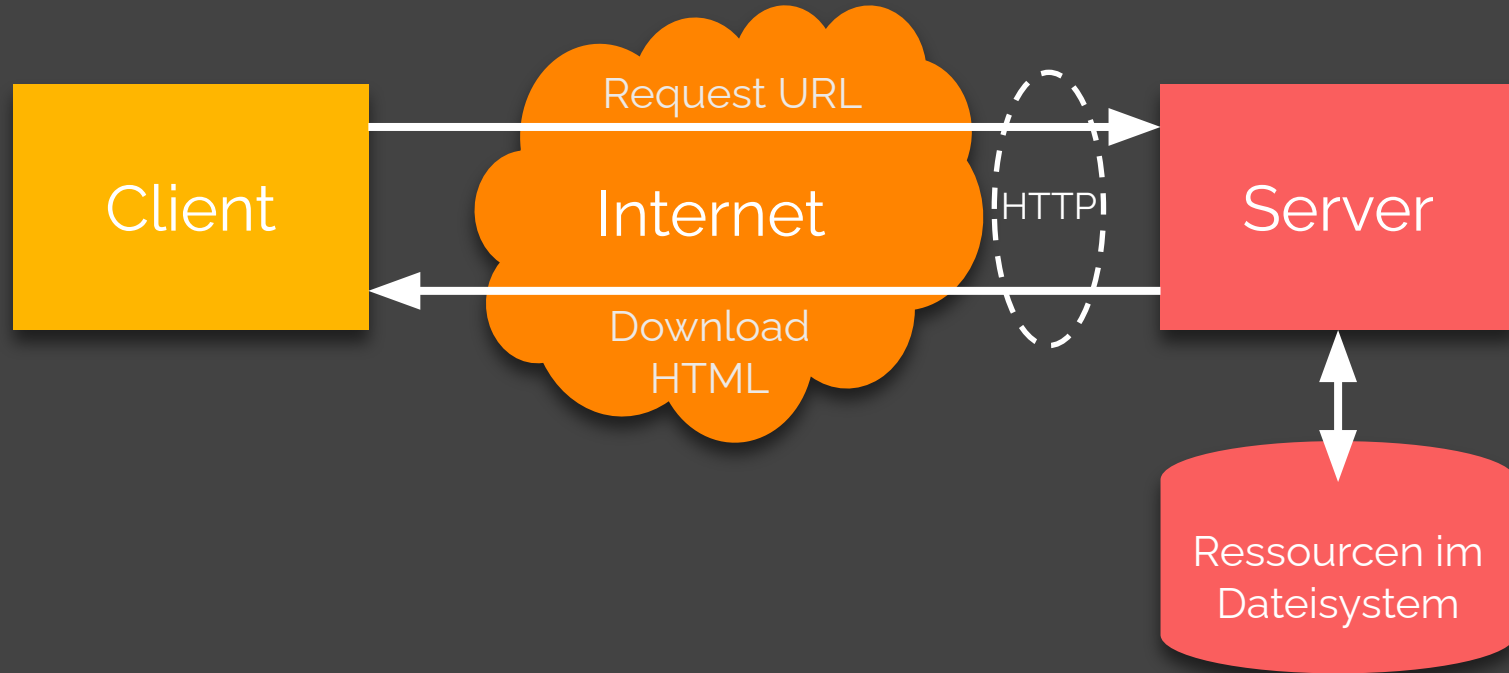
- `https://` Schema
- `www.fh-ooe.at` Host
- `/` Pfad (Wurzelverzeichnis)

Port ist implizit :443 für HTTPS.

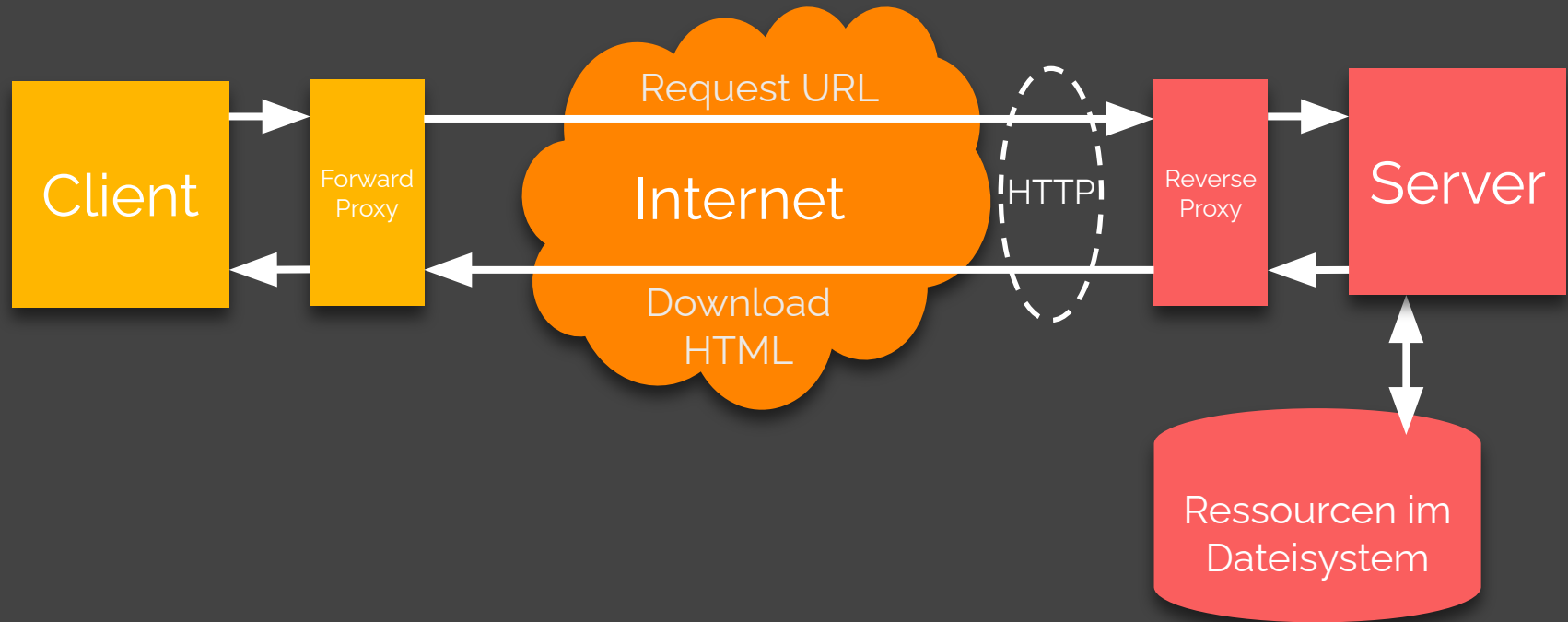
Das Client-Server-Modell

Kommunikationsablauf in einem Netzwerk

Remember this?



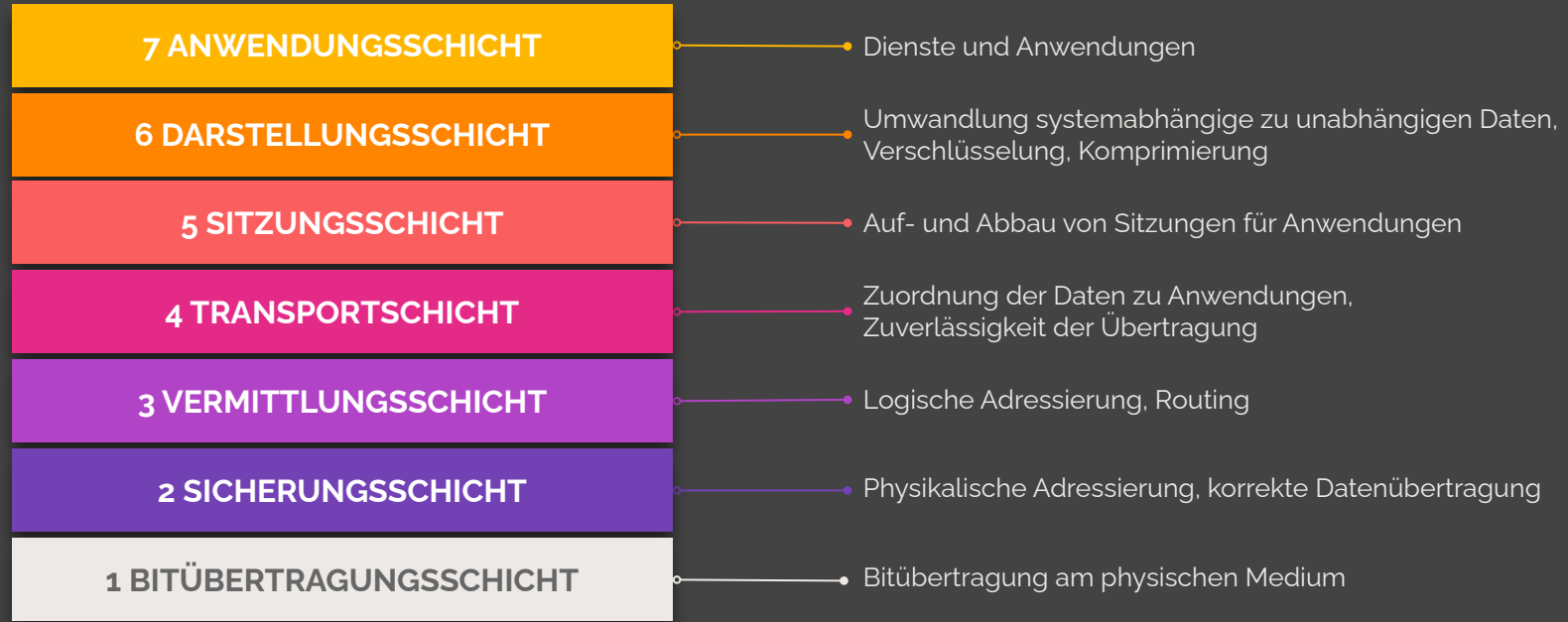
Nicht immer direkt: Proxy



Schichtenmodelle

Kommunikation im Netz abbilden: OSI und TCP/IP

OSI-Modell



TCP/IP -Modell



OSI	TCP/IP	Protokolle	Dateneinheit
7 Anwendung	Anwendung	HTTP(S), SMTP, DNS, SSH	Nachricht / Daten
6 Darstellung			
5 Sitzung			
4 Transport	Transport	TCP, UDP	Segment / Datagramm
3 Vermittlung	Internet	IP, ICMP	Paket
2 Sicherung	Netzzugang	Ethernet, WLAN, ARP Kupfer, Glasfaser, Funk	Frame Bit
1 Bitübertragung			

Encapsulation (Web Req.)



HTTP

HTTP-Header + HTTP-Body (HTML, JSON, ...)

TCP

TCP
Header

HTTP-Nachricht als Payload

IP

IP Header

TCP-Segment als Payload

Ethernet

Ethernet
Header

IP-Paket als Payload

CRC

DNS

Vom Namen zur IP-Adresse



FH OÖ Hostname

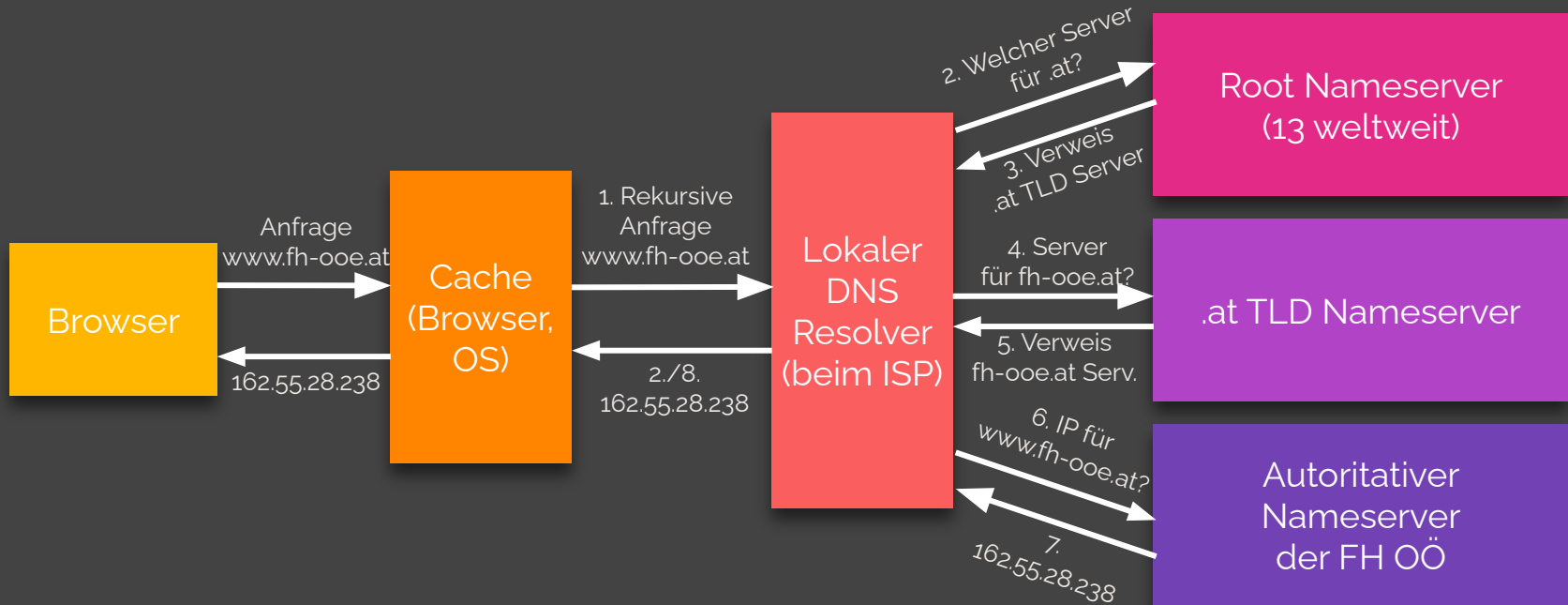
`www.fh-ooe.at` Fully Qualified Domain Name (FQDN)

- `at` Top-Level-Domain (TLD)
- `fh-ooe` Second-Level-Domain (SLD)
- `www` Hostname, auch Subdomain

Domain-Namen sind für Menschen gemacht, Rechner werden mit `IP-Adressen` adressiert.

Das `DNS (Domain Name System)` erledigt diese Zuordnung.

DNS Namensauflösung



Netzwerkschicht

IP und lokale Zustellung



IP-Adressen

162.55.28.238

IPv4 Adresse

$4 \cdot 8 = 32$ Bit lang (ca. 4,3 Milliarden Adressen nutzbar)

2a01:04f8:2190:2103:d64a:3471:c57f:8d4c

IPv6 Adresse

$8 \cdot 16 = 128$ Bit lang (ca. 340 Sextillionen Adressen möglich)

Das ist die Zahl: 340,282,366,920,938,463,463,374,607,431,768,211,456



IP-Adresse

Beispiel Lokale Adresse: 192.168.0.17

Aufbau

Subnetzmaske bestimmt **Netzanteil** und **Hostanteil** einer IP:

Z.B. 255.255.255.0: 192.168.0.17

Moderne CIDR (Classless Inter Domain Routing) Schreibweise:

192.168.0.17/24

Ganze Adresse hat 32 Bit, 24 sind der Netzanteil (8 bleiben für Host)

IP Beispiele



192.168.0.17/24

- Subnetzmaske: 255.255.255.0
- $32-24 = 8$ Bit für Hosts
- $2^8 = 256$ mögliche Hosts
- 192.168.0.0 bis 192.168.0.255
- Aber: 192.168.0.0 ist die Netz-ID, 192.168.0.255 ist die Broadcast-IP. Diese sind reserviert und fallen weg (-2 Regel)
- 254 mögliche Hosts: 192.168.0.1 bis 192.168.0.254

10.42.7.200/16

- Subnetzmaske: 255.255.0.0
- $32-16 = 16$ Bit für Hosts
- $2^{16} - 2 = 65.534$ mögliche Hosts
- 10.42.0.1 bis 10.42.255.254
- 10.42.0.0 ist die Netz-ID 10.42.255.255 ist die Broadcast-IP.

IP Kommunikation



Rechner im **selben Subnetz** können miteinander auf Ethernet-Ebene kommunizieren.

Rechner in **unterschiedlichen** Subnetzen müssen über einen **Router** kommunizieren.

Der Router des eigenen Subnetzes wird als **Gateway** bezeichnet.

IP-Pakete können in **3 Modi** verschickt werden:

1. **Unicast**: 1 Absender schickt an einen Empfänger.
2. **Broadcast**: Ein Absender schickt an alle Rechner im Subnetz über die Broadcast-IP.
3. **Multicast**: Ein Paket ergeht an eine Gruppe von Rechnern.

Broadcasts werden von Routern nie in andere Subnetze weitergeleitet (Chaos).

Private Adressen & NAT



Bestimmte IPv4-Bereiche sind als **private Adressen** gekennzeichnet und werden **nicht öffentlich geroutet**:

- 10.0.0.0/8: 10.0.0.0 bis 10.255.255.255
- 172.16.0.0/12: 172.16.0.0 bis 172.31.255.255
- 192.168.0.0/16: 192.168.0.0 bis 192.168.255.255

Damit Pakete von Rechnern in privaten Netzen trotzdem Pakete verschicken und empfangen kann, kommt **NAT (Network Address Translation)** zum Einsatz.

Der Router am Übergang zum Internet **ersetzt in ausgehenden IP-Paketen die private Quell-IP durch seine öffentliche IP-Adresse**, nach Außen sichtbare und merkt sich die Ports. Bei einer Antwort wird die Ersetzung wieder rückgängig gemacht.

Darum funktioniert IPv4 (trotz der viel zu wenigen Adressen) heute immer noch.

DHCP: Eine IP beziehen



Wie kommt ein Rechner in einem Netzwerk eine eigene IP-Adresse?

Durch das **Dynamic Host Configuration Protocol (DHCP)**. Folgende Schritte sind involviert:

1. Rechner verbindet sich mit LAN/WLAN
2. Er sendet einen DHCP-Request per Broadcast ins lokale Netz.
3. Ein DHCP-Server antwortet.
4. Die Antwort enthält ein Datenpaket mit
 - a. einer freien IP-Adresse (z.B. **192.168.0.17**)
 - b. der Subnetzmaske (z.B. 255. 255. 255. 0)
 - c. der IP-Adresse des Gateways (z.B. **192.168.0.1**)
 - d. der IP-Adresse mindestens eines DNS-Resolvers (z.B. **192.168.0.1** oder **212.33.36.155**)

ARP: Von IP zu MAC



Zurück zum Web Request. Der Rechner kennt jetzt von `https://www.fh-ooe.at/`

- URL mit Hostnamen und IP-Adresse durch DNS: `162.55.28.238`
- Seine eigene IP-Adresse: z.B. `192.168.0.17`

Der Webserver ist offensichtlich nicht im selben Subnetz. Die Pakete müssen zum **Gateway** geschickt werden.

Aber: Ethernet (zuständig für die physische Zustellung der Pakete) kennt keine IP-Adressen, nur Hardware-Adressen, genannt **MAC-Adressen (Media Access Control)**.

Der Laptop braucht also die MAC-Adresse des Gateways. Diese bekommt er mit dem **Address Resolution Protocol (ARP)**. Er schickt einen ARP-Broadcast ins Netz und fragt, wer hat die Adresse `192.168.0.1`?

Der Gateway antwortet mit seiner MAC-Adresse: `00:0e:7f:27:a8:59`

Ethernet & IP: Zustellung



Ethernet ist für die physische Zustellung der Daten im lokalen Netzwerk zuständig. Die Dateneinheiten heißen Frame.

Ein Ethernet-Frame besteht aus einem Header (mit Quell- und Ziel-MAC-Adresse) und dem darin gekapselten IP-Paket.

Der Laptop stellt nun mehrere Frames zusammen (der gekapselte Web-Request) und verschickt diese. Die Netzwerkkarte sendet sie zum Gateway. Switches im lokalen Netzwerk leiten die Frames dabei an den richtigen Port weiter.

Der Gateway entfernt den Ethernet-Header, liest die Ziel IP 162.55.28.238 aus und leitet das enthaltene IP-Paket anhand von Routing-Tabellen an andere Router weiter. Bei jedem Hop wird der Ethernet-Frame neu gebaut, das IP-Paket bleibt immer erhalten. Ein Time-to-Live (Zähler) stellt sicher, dass Pakete nach zu vielen Hops gelöscht werden.

So reisen die IP-Pakete bis zum Webserver. Aber kommt dort auch alles richtig an?

Transportschicht

TCP (und UDP)

TCP: Sicherer Transport



IP ist auf der Netzwerkschicht ein verbindungsloses und unzuverlässiges (best-effort) Protokoll.

- IP garantiert nicht, dass das Paket ankommt.
- IP garantiert keine korrekte Reihenfolge.
- IP prüft nicht, ob Daten korrupt sind.

Hier setzt **TCP** an: Es baut auf dem unzuverlässigen IP-System eine **zuverlässige Verbindung** auf, es sorgt für die **korrekte Reihenfolge**, **fordert verlorene Pakete** erneut an und **steuert die Datenrate**.

Zum Vergleich: **UDP** ist das zweite Protokoll auf der Transportschicht:

- verschickt einzelne Datagramme **ohne Verbindungsaufbau** und **ohne Zustellgarantie**.
- Deutlich **schlanker** und eignet sich für kurze Anfragen (DNS), Echtzeitanwendungen wie VoIP und Video-Streaming.
- Überall dann okay, wo verlorene Pakete nichts ausmachen.

Ports: Dienste trennen



Während IP mit der IP-Adresse nur Rechner adressiert, kann TCP (und auch UDP) mit **Ports** mehrere Dienste auf einem Rechner unterscheiden und die Daten gezielt zustellen. Dadurch wird **Multiplexing** möglich (mehrere Datenströme an einen Rechner).

Bekannte Beispiele:

- FTP: 20, 21
- SSH: 22
- SMTP: 25
- DNS: 53
- DHCP: 67, 68
- HTTP: 80
- HTTPS: 443

Insgesamt gibt es 65.535 verfügbare Ports. Ports unter 1024 sind reserviert.

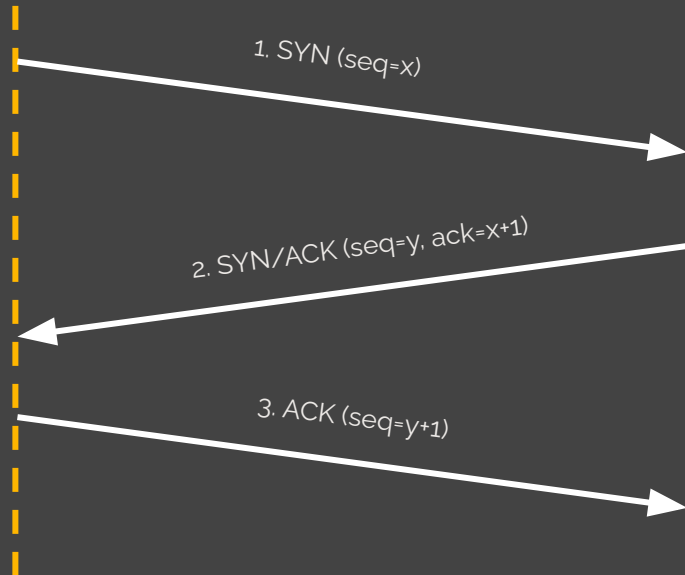
Bei einer TCP-Verbindung wählt sich der Client auf seiner Seite einen Port (49152–65535).

TCP-Verbindung



Client

Server



Nach dem **Handshake** können Daten geschickt werden.

Der Webrequest wird in **TCP-Segmente** zerteilt. Diese enthalten im Header die Sequenznummer.

Jedes TCP-Segment kommt in ein **IP-Paket** mit Quell- und Ziel-IP.

Jedes IP-Paket kommt in einen **Ethernet-Frame** mit Quell- und Ziel-MAC.

Auf der Gegenseite passiert das Umgekehrte. **TCP sortiert die Segmente** und fordert fehlende nochmals an.

Encapsulation (Web Req.)



HTTP

HTTP-Header + HTTP-Body (HTML, JSON, ...)

TCP

TCP
Header

HTTP-Nachricht als Payload

IP

IP Header

TCP-Segment als Payload

Ethernet

Ethernet
Header

IP-Paket als Payload

CRC

Sichere Übertragung

HTTPS und TLS



HTTPS

HTTPS ist kein eigenes Protokoll. Es ist HTTP, das durch TLS (Transport Layer Security) geschützt wird. TLS schiebt sich zwischen TCP und HTTP und sorgt für ...

Vertraulichkeit

Übertragenen Daten werden verschlüsselt, sodass Dritte sie nicht mitlesen können.

Integrität

Änderungen an den Daten während der Übertragung werden bemerkt.

Authentizität

Der Client kann sich sicher sein, tatsächlich mit dem Server zu sprechen, dessen Name im URL steht

TLS-Verbindung



In einem **TLS-Handshake** handeln Client und Server einen gemeinsamen Schlüssel aus.

1. Mit **ClientHello** werden **TLS-Version**, **Verschlüsselungsverfahren** und eine Zufallszahl mitgeschickt.
2. Mit **ServerHello** wählt der Server die TLS-Version aus, wählt ein Verschlüsselungsverfahren und schickt sein Zertifikat.
3. Der Client überprüft das **Zertifikat**. Ist es gültig? Wurde es von einer Certification Authority (CA) ausgestellt?
4. Dann findet ein Schlüsselaustausch und leiten einen symmetrischen Sitzungsschlüssel ab.
5. Mit **Finished** wird signalisiert, dass jetzt verschlüsselt kommuniziert wird.

Bekannte CAs:

- Let's Encrypt
- DigiCert
- GlobalSign

Browser liefern eine Liste an **vertrauenswürdigen CAs** (Trust Store) mit, um Zertifikate überprüfen zu können.

HTTP

Das Protokoll des Web

HTTP-Request



GET / HTTP/1.1

Host: www.fh-ooe.at

User-Agent: Mozilla/5.0 (...) Firefox/126.0

Accept: text/html,application/xhtml+xml,*/*;q=0.8

Accept-Language: de-AT,de;q=0.9,en;q=0.8

[Leerzeile]

Methode, Pfad, HTTP-Version

Host

Identifikation des Browsers

Was der Browser kennt

Bevorzugte Sprache

GET-Requests fordern nur Inhalte an und haben nur Header und keinen Message-Body, daher kommt nach der Leerzeile (Trenner zum Body) nichts mehr.

Nachdem der Browser diesen Request erhalten hat, antwortet er (Response).

HTTP-Response



HTTP/1.1 200 OK

Date: Mon, 09 Apr 2026 06:50:00 GMT

Server: Apache/2.4.58 (Ubuntu)

Content-Type: text/html; charset=UTF-8

Content-Length: 11321

<!DOCTYPE html>

<html lang="de">

...

</html>

HTTP-Version, Statuscode und Reason

Datum der Antworterzeugung am Server

Identifikation des Servers

MIME-Type und Zeichensatz der Response

Länge des Bodys

Der Body, also der eigentlich Inhalt der Response. Hier ein HTML-Dokument.



HTTP-Methoden

GET

Daten vom Server anfragen (lesen).

POST

Daten an den Server senden (schreiben).

PUT

Daten am Server ersetzen (aktualisieren).

DELETE

Daten am Server löschen.

PATCH

Daten am Server teilweise ändern.

HEAD

Daten anfragen, aber nur Header kommen retour.

OPTIONS

Welche Methoden sind erlaubt?



Wichtige Statuscodes

Jede Response enthält einen Statuscode.

1xx: Informationen	2xx: Erfolgreich	3xx: Umleitung	4xx: Client-Fehler	5xx: Server-Fehler
100 CONTINUE	200 OK	301 MOVED PERMANENTLY	400 BAD REQUEST	500 INTERNAL SERVER ERROR
	201 CREATED	304 NOT MODIFIED	401 UNAUTHORIZED	501 NOT IMPLEMENTED
	204 NO CONTENT		403 FORBIDDEN	502 BAD GATEWAY
			404 NOT FOUND	503 SERVICE UNAVAILABLE
			406 NOT ACCEPTABLE	



HTTP- Versionen

HTTP existiert seit den 1990ern, wird aber weiterhin aktiv entwickelt.

Die am weitesten verbreitete Version ist momentan HTTP/2.

HTTP/1.0 (1996)

Pro Request eine TCP-Verbindung, wird nach der Response geschlossen.

HTTP/1.1 (1999)

Verbindung bleibt offen (keep-alive), Pipelining der Requests. Host-Header verpflichtend.

HTTP/2 (2015)

Binäres Nachrichtenformat, Header-Kompression, Multiplex, Server Push

HTTP/3 (2022)

Statt TCP wird UDP (QUIC-Protokoll) verwendet. Extrem performant.

HTTP ist zustandslo s



Jeder Request-Response-Zyklus wird **isoliert betrachtet**. Der Server „erinnert“ sich nicht, wenn ein Client gerade zuvor schon einmal angefragt hat. Daten müssen daher mit eigenen Mechanismen über mehrere Seitenaufrufe hinweg erhalten werden.

Recap

Der Gesamt Ablauf eines Web Requests nochmals im Überblick



Web-Reques

t Involvierte Protokolle und Dienste:

- DHCP (OSI-Schicht 7)
- DNS (OSI-Schicht 7)
- ARP (OSI-Schicht 2)
- HTTP (OSI-Schicht 7)
- TLS (OSI-Schicht 5)
- TCP (OSI-Schicht 4)
- IP (OSI-Schicht 3)
- Ethernet (OSI-Schicht 2)



← Schwerpunkt in WET2VO ab Vorlesung 2.



Nächstes Mal in WET2VO

PHP-Grundlagen Teil 1

Credits



- S. 11: Webseite der FH Oberösterreich: <https://www.fh-ooe.at/>